	カテゴリ	機能詳細	Servlet (Controller)
また	**A. 認証・初期**	ログイン / ホーム	User_LoginServlet HomeServlet GiveHomeServlet
のな	**B. ユーザー管理**	マイページ / 更新	MypageServlet Profile_UpdateTransitionServlet Profile_UpdateServlet Profile_DeleteServlet
たば	**C. 教材検索・登録**	検索 / 登録 / 評価	TextsSearchServlet TextinforegisterServlet Textinfodelete MarkingServlet
まつ	**D. 提供・推薦**	推薦リスト / 提供管理	SuggestedListServlet ProvisionListServlet Providerconpservlet
なか	**E. 取引・履歴**	取引進行 / 履歴表示	TradeHomeServlet TransactionProviderservlet TransactionHistoryServlet
わご	**F. 管理者機能**	登録・更新・削除 ユーザー審査	AdminServlet AdminHomeServlet AdminActionServlet AdminRegistrationServlet AdminUpdateServlet AdminDeleteServlet userAppoveServlet UserListServlet
いの	**G. 共通基盤**	接続・状態	-

Model / Logic / DAO	JSP (View)
LoginLogic GiveHomeLogic	giveHome.jsp givehomebtn.jsp
MypageLogic ProfileDeleteLogic UserDao User.java	Profile.jsp ProfileUpdate.jsp
TextSearchLogic MarkingLogic TextDao SubjectDao Textbook.java Subject.java SearchResult SubjectTextbook	textsearch.jsp textinforegister.jsp
SuggestedListLogic ProvisionListLogic SuggestedTextbookDao SuggestedInfoDao ProvisionDao SuggestedInfo Provision	Provider.jsp
TransactionHistoryLogic TransactionDao TextTradeDao TextTrade.java TradePartnerView	(取引画面共通)
AdminSearchListLogic AdminListLogic AdminListRegistrationLogic AdminListDeleteLogic AdminListRetireLogic AdminListConformLogic	administrator_updated.jsp administrator_registration.jsp administrator_List.jsp administrator_Delete.jsp userApprove.jsp userList.jsp
DbConnect.java	-

Status.java

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TradePartnerView
プログラミング担当	**中村**
簡単な説明	取引相手に関する情報を保持するためのモデルクラスです。進行中の取引一覧画面において、取引番号や教科書名に加え、相手の氏名や連絡先（メールアドレス）を表示・管理するために使用します。

テストNo	TradePartnerView	TradePartnerView	TradePartr	TradePartnerView	TradePartnerView05
テスト担当者	中村	中村	中村	中村	
結果					
全てのフィールドに正常値をセット	Y	-	-	Y	
emailAddress に不正な形式の文字列	-	Y	-	-	
fullName が空文字 ("")	-	-	Y	-	
取引番号(tradingNumber)に 0 をセット	-	-	-	Y	
期待：Getterからセットした値が返る	X	X	X	X	
期待：JSPでの表示時にエラーが出ない	X	X	X	X	
期待：メール送信(mailto)に使用可能	X	X	-	-	

対象となるソースコード

```
public class TradePartnerView {
  private int tradingNumber;   // 取引番号
    private String textbookName;  // 教科書名
    private int userId;        // 取引相手（希望者）のID
    private String fullName;      // 取引相手の名前
  private String emailAddress; // 取引相手のメールアドレス

    // 各項目の getter / setter （省略）
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TextTrade.java
プログラミング担当	**中村**
簡単な説明	取引に関するデータを一括して保持するためのモデルクラスです。データベースの基本項目に加え、画面表示に使用する結合結果（氏名や教科書名、メールアドレスなど）を格納・管理します

テストNo	TextTrade.java01	TextTrade.java02	TextTrade.	TextTrade.java04	TextTradeDao05
テスト担当者	中村	中村	中村	中村	
結果					
全てのフィールドに正常値をセット	Y	-	-	-	
String型（comment等）にnullをセット	-	Y	-	-	
Timestamp型に現在時刻をセット	Y	-	Y	-	
JOIN用フィールド(textbookName等)を使用	-	-	-	Y	
期待：Getterからセットした値が返る	X	X	X	X	
期待：NullPointerExceptionが発生しない	X	X	X	X	
期待：型変換エラーが発生しない	X	-	X	-	

対象となるソースコード

```
public class TextTrade {
  private int tradingNumber;    // 取引番号
    private int provisionNumber;    // 提供番号
  private int suggestedNumber;  // 希望番号
    private int textbookNumber;     // 教科書番号
  private String comment;       // コメント
    private Timestamp tradingDatetime; // 取引日時
    private int statusId;           // ステータスID

    // 表示用（JOIN結果）
    private String emailAddress;    // メールアドレス
    private String textbookName;    // 教科書名
  private String fullName;      // 氏名

    // getter / setter は省略
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TextTradeDao
プログラミング担当	**中村**
簡単な説明	データベース操作を担い、取引情報の1件検索、ステータスを完了（completed）にする更新処理、および提供者ごとの進行中取引一覧の取得など、取引に関する業務ロジックに必要なデータを抽出・反映します。

テストNo	TextTradeDao01	TextTradeDao02	TextTradeI	TextTradeDao04	TextTradeDao05
テスト担当者	中村	中村	中村	中村	
結果					
findOngoingByProvider を実行	Y	-	-	-	
completeTrade を実行	-	Y	Y	-	
findById を実行	-	-	-	Y	
対象の取引が存在する	Y	Y	Y	N	
現在の status_id が 3 (ongoing) である□	Y	Y	N	-	
期待：Listのサイズが1以上返る	X	-	-	-	
期待：戻り値が true (更新成功)	-	X	-		
期待：戻り値が false (更新対象なし)		-	X	-	
期待：戻り値が null				X	

対象となるソースコード

```
public class TextTradeDao {
  // DB接続の取得
    public TextTradeDao() throws SQLException {
        this.conn = DbConnect.getConnection();
    }

    // 取引の1件検索
    public TextTrade findById(int tradingNumber) throws SQLException {
    String sql = "SELECT * FROM textbook_trading_info WHERE trading_number = ?";
        // 実行結果があればmapメソッドでモデルに変換、なければnullを返す
        return rs.next() ? map(rs) : null;
    }

    // 取引完了処理 (ongoing -> completed)
  public boolean completeTrade(int tradingNumber) throws SQLException {
        String sql = "UPDATE textbook_trading_info SET status_id = ? WHERE trading_number = ? AND status_id = ?";
    return ps.executeUpdate() == 1; // 更新件数が1件なら成功
    }

    // 提供者ごとの進行中取引リスト取得
    public List<TradePartnerView> findOngoingByProvider(int providerUserId) throws SQLException {
    String sql = "SELECT ... FROM tt JOIN si JOIN u JOIN t JOIN st WHERE tt.status_id = 3 AND ...";
        // 取得したレコードをListに格納して返す
        return list;
    }
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TransactionDao
プログラミング担当	**中村**
簡単な説明	引数として受け取ったユーザIDを解釈し、DAOを介して受領者IDに紐づく取引履歴の一覧を取得する業務ロジックを実行します。

テストNo	TransactionDao01	TransactionDao02	Transactio	TransactionDao04	TransactionDao05
テスト担当者	中村	中村	中村	中村	
結果					
DbConnect.getConnection()に成功する	Y	Y	N	Y	
指定したuserIdのデータがDBに存在する	Y	N	-	Y	
JOIN先の関連テーブルにデータが揃っている	Y	-	-	N	
戻り値がList<TransactionView>である	X	X	-	-	
Listのサイズがレコード数と一致する	X	-	-		
SQLのORDER BY通り降順で並んでいる		-	X	-	

対象となるソースコード

```
// --- TransactionHistoryLogic (業務ロジック) ---
public class TransactionHistoryLogic {
    public List<TransactionView> execute(int userId) throws SQLException {
        // DAOを呼び出し、受領者IDに紐づく取引履歴を返す
    return new TransactionDao().findByRecipientUserId(userId);
    }
}

// --- TransactionProviderservlet (サーブレット) ---
                            #NAME?
public class TransactionProviderservlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws Exception {
        // パラメータ取得とエンコードを行い、mailtoリンクを生成
        String to = request.getParameter("email");
```

```
        String body = request.getParameter("body");
        String mailto = "mailto:" + to + "?subject=...&body=" + URLEncoder.encode(body, "UTF-8");

    // JSでメーラーを起動し、2秒後に元の画面(Servlet)へ戻る
        response.getWriter().println("<script>window.location.href='" + mailto + "';</script>");
    }
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TransactionHistoryLogic
プログラミング担当	**中村**
簡単な説明	ServletとしてHTTPリクエストを受け取り、セッションによるログイン状態を判定。正常時は業務ロジックで取引履歴を取得し、未ログイン時はログイン画面へ、正常時は履歴画面への遷移先を決定します。

テストNo	TransactionHistory	TransactionHistory	Transactio	TransactionHistoryLogic04
テスト担当者	中村	中村	中村	中村
結果				
有効なuserIdが渡される	Y	Y	Y	N
DB接続（TransactionDao）が正常	Y	Y	N	Y
対象ユーザーの取引履歴が存在する	Y	N	-	-
Logic.executeが正常終了する	Y	Y	-	N
List<TransactionView>が返る	X	X	-	-
戻り値のListのサイズが1以上	X	-	X	
戻り値のListのサイズが0 (空)		X		X

対象となるソースコード

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws Exception {
   User loginUser = (User) request.getSession().getAttribute("loginUser");

    // 【分岐1】未ログイン時：ログイン画面へリダイレクト
   if (loginUser == null) {
        response.sendRedirect("WEB-INF/jsp/login.jsp");
    return;
    }

    try {
    // 【分岐2】正常系：履歴を取得してJSPへフォワード
        List<TransactionView> list = new TransactionHistoryLogic().execute(loginUser.getUserId());
        request.setAttribute("transactionList", list);
        request.getRequestDispatcher("WEB-INF/jsp/transactionHistory.jsp").forward(request, response);

    } catch (SQLException e) {
        // 【分岐3】異常系：サーバーエラーを送出
    throw new ServletException(e);
    }
}

protected void doPost(HttpServletRequest request, HttpServletResponse response) throws Exception {
    doGet(request, response); // POST時もGETと同じ処理
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TransactionHistoryServlet
プログラミング担当	**中村**
簡単な説明	ServletとしてHTTPリクエストを受け取り、ログイン状態の確認と入力値（ユーザID）に基づいた業務ロジック（取引履歴取得）を呼び出し、遷移先を決定します

テストNo	TransactionHistory	TransactionHistory	Transactio	TransactionHistory04
テスト担当者	中村	中村	中村	中村
結果				
メソッド doGet	Y	N	Y	Y
メソッド doPost	N	Y	N	N
ログイン状態（loginUser有）	Y	Y	N	Y
Logic.executeが正常終了する	Y	Y	-	N
transactionHistory.jspにフォワード	X	X	-	-
login.jspにリダイレクト		-	X	

対象となるソースコード

```
                    #NAME?
public class TransactionHistoryServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws Exception {
        User loginUser = (User) request.getSession().getAttribute("loginUser");

        // 【判定】未ログインならログイン画面へ
    if (loginUser == null) {
            response.sendRedirect("WEB-INF/jsp/login.jsp");
      return;
        }

        try {
            // 【処理】履歴を取得してJSPへ渡す
            List<TransactionView> list = new TransactionHistoryLogic().execute(loginUser.getUserId());
            request.setAttribute("transactionList", list);
      request.getRequestDispatcher("WEB-INF/jsp/transactionHistory.jsp").forward(request, response);
    } catch (SQLException e) {
      throw new ServletException(e); // 【例外】エラー送出
        }
```

```
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws Exception {
        doGet(request, response); // GETと同じ処理を実行
    }
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TransactionProviderservlet
プログラミング担当	**中村**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値を解釈して業務ロジック（取引一覧取得やメール連携処理）を呼び出し、遷移先を決定します。

テストNo	TransactionProvid	TransactionProvid	Transactio	TransactionProviders04
テスト担当者	中村	中村	中村	中村
結果				
メソッド doGet	Y	Y	N	Y
メソッド doPost	N	N	Y	N
ログイン状態（セッション有）	Y	Y	Y	N
DB接続が正常である	Y	N	-	Y
Provider.jspが存在する	Y	Y	N	-
Provider.jspにフォワードされる	X	X	-	-
エラーメッセージ(errorMessage)がセットされる		X		
mailtoリンク（JS）が出力される			X	

対象となるソースコード

```
                    #NAME?
public class TransactionProviderservlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws Exception {
        // メーラーを起動し、2秒後に自分自身のGETへ遷移
        String mailto = "mailto:" + req.getParameter("email") + "?subject=...&body=" + URLEncoder.encode(req.getParameter("body"), "UTF-8");
        res.getWriter().println("<script>");
        res.getWriter().println("window.location.href='" + mailto + "';");
    res.getWriter().println("setTimeout(function(){ window.location.href='TransactionProviderservlet'; }, 2000);");
        res.getWriter().println("</script>");
    }

    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws Exception {
        try {
      // ログイン中の提供者の進行中取引を取得
            User user = (User) req.getSession().getAttribute("loginUser");
            req.setAttribute("tradeList", new TextTradeDao().findOngoingByProvider(user.getUserId()));
```

```
        } catch (SQLException e) {
      req.setAttribute("errorMessage", "データ取得エラー");
        }
        req.getRequestDispatcher("WEB-INF/jsp/Provider.jsp").forward(req, res);
  }
}
```

```
    }
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TradeHomeServlet.java
プログラミング担当	**中村**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値の有無に関わらず、センターホーム画面（centerHome.jsp）への遷移先を決定します。

テストNo	TradeHome01	TradeHome02	TradeHon	TradeHome04
テスト担当者	中村	中村	中村	中村
結果				
メソッド doGet	Y	N	Y	-
メソッド doPost	N	Y	N	-
ログイン状態（セッション有）	Y	Y	N	-
centerHome.jspが存在する	Y	Y	N	-
centerHome.jspにフォワードされる（遷移する）	X	X	X	-

対象となるソースコード

```
package servlet;

import java.io.IOException;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/TradeHomeServlet")
public class TradeHomeServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
```

```
        req.getRequestDispatcher("/WEB-INF/jsp/centerHome.jsp")
            .forward(req, res);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {

        // POSTでもGETと同じ処理
        doGet(req, res);
    }
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	User_LoginServlet
プログラミング担当	**又間**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値を解釈して業務ロジックを呼び出し、遷移先を決定します。

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04
テスト担当者	太郎	次郎	三郎	四郎
結果	OK	OK	OK	OK
loginLogic = new LoginLogic()	Y	N	Y	Y
User loginUser = loginLogic.e	Y		N	Y
loginUser != null	Y			N
e.printStackTrace(); response.sendError(HttpServl return;	-	X	-	-
e.printStackTrace(); response.sendError(HttpServl	-	-	X	-
request.setAttribute("loginErr RequestDispatcher dispatcher dispatcher.forward(request, r	-	-	-	X

対象となるソースコード

```
try {
        loginLogic = new LoginLogic();
    } catch (Exception e) {
```

```
    e.printStackTrace();
    response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
    return;
}

try {
    User loginUser = loginLogic.execute(fullName, password);

    if (loginUser != null) {
        HttpSession session = request.getSession();
        session.setAttribute("loginUser", loginUser);

        RequestDispatcher dispatcher = request.getRequestDispatcher("WEB-INF/jsp/Home.jsp");
        dispatcher.forward(request, response);
    } else {
        request.setAttribute("loginError", "氏名またはパスワードが違います");
        RequestDispatcher dispatcher = request.getRequestDispatcher("WEB-INF/jsp/user_Login.jsp");
        dispatcher.forward(request, response);
    }

} catch (SQLException e) {
    e.printStackTrace();
    response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	textsearch.jsp
プログラミング担当	**田畑**
簡単な説明	教科書検索を行う画面

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04	waribiki-0	waribiki-0	waribiki-0	waribiki-08
テスト担当者	太郎	次郎	三郎	四郎	四郎	四郎	四郎	四郎
結果	OK	OK	OK	OK	OK	OK	OK	OK
tList == null	Y	N	Y	Y	Y	Y	Y	Y
t.getTextbookNumber() == textbookNumber	Y	N	N	Y	Y	Y	Y	Y
pList != null && pList.isEmpty() && request.getParameter("keyword") !	Y	N	N	N	Y	Y	Y	Y
sList != null && !sList.isEmpty()	Y	N	N	N	N	Y	Y	Y
pList != null && !pList.isEmpty()	Y	N	N	N	N	N	Y	N
prov.getTextbookPhoto() != null && !prov.getTextbookPhoto().isEmpty()	Y	N	N	N	N	N	N	N
return null;	X	-	X	X	X	X	X	X
return t;	X	-	-	X	X	X	X	X
該当する情報は見つかりませんでした。	X	-	-	-	X	X	X	X
for (Subject sub : sList) { %> tr> td><input type="checkbox" class="subject-check" value="<%=sub.getSubjectsNumber()%>"></td> td><%=sub.getSubjectsName()%></td> td><%=sub.getSubjectsDetail()%></td> /tr> %	X	-	-	-	-	X	X	X
科目情報が見つかりません。	-	X	X	X	-	-	-	-

for (Provision prov : pList) {
extbook book = getBook(prov.getTextbookNumber(), tList);
%>
div class="info-panel" data-subject="<%=prov.getSubjectsNumber()%>">
p>
教科書情報 <input type="checkbox" name="provisionNumber"
value="<%=prov.getProvisionNumber()%>"> 欲しい！
/p>
	X	-	-	-	-	-	X	X

div class="info-box">教科書名：<%=(book != null) ? book.getTextbookName() : "不明"%> 教科書状態：<%=prov.getTextbookDetail()%> 写真：								
<img rc="<%=request.getContextPath() + "/" + prov.getTextbookPhoto()%>" tyle="max-width: 100%; height: auto;">	X	-	-	-	-	-	-	X
教科書情報 div class="info-box">提供されている教科書はありません。	X	X	X	X	X	X	X	-

対象となるソースコード

```
if (tList == null)
            return null;
if (t.getTextbookNumber() == textbookNumber)
                return t;
<%
        if (pList != null && pList.isEmpty() && request.getParameter("keyword") != null
                && !request.getParameter("keyword").isBlank()) {
        %>
        <div style="color: red; font-weight: bold; margin-top: 10px;">
            該当する情報は見つかりませんでした。</div>
        <%
        }
```

```
<%
                if (sList != null && !sList.isEmpty()) {
                    for (Subject sub : sList) {
                %>
                <tr>
                    <td><input type="checkbox" class="subject-check"
                        value="<%=sub.getSubjectsNumber()%>"></td>
                    <td><%=sub.getSubjectsName()%></td>
                    <td><%=sub.getSubjectsDetail()%></td>
                </tr>
                <%
                }
                } else {
                %>
                <tr>
                    <td colspan="3">科目情報が見つかりません。</td>
                </tr>
                <%
                }
<%
            if (pList != null && !pList.isEmpty()) {
                for (Provision prov : pList) {
                    Textbook book = getBook(prov.getTextbookNumber(), tList);
            %>
            <div class="info-panel" data-subject="<%=prov.getSubjectsNumber()%>">
                <p>
                    教科書情報 <input type="checkbox" name="provisionNumber"
                        value="<%=prov.getProvisionNumber()%>"> 欲しい！
                </p>

                <div class="info-box">
                    教科書名：<%=(book != null) ? book.getTextbookName() : "不明"%><br>
                    教科書状態：<%=prov.getTextbookDetail()%><br> 写真：
                    <%
                if (prov.getTextbookPhoto() != null && !prov.getTextbookPhoto().isEmpty()) {
                %>
                    <div class="image-area">
                        <img
                            src="<%=request.getContextPath() + "/" + prov.getTextbookPhoto()%>"
                            style="max-width: 100%; height: auto;">
                    </div>
                    <%
                    } else {
                    %>
                    なし
                    <%
                    }
                    %>

                </div>
            </div>
            <%
            }
            } else {
            %>
            <div class="info-panel">
```

```
    <p>教科書情報</p>
    <div class="info-box">提供されている教科書はありません。</div>
</div>
<%
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	textinforegister.jsp
プログラミング担当	**田畑**
簡単な説明	教科書情報を追加する画面

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04
テスト担当者	太郎	次郎	三郎	四郎
結果	OK	OK	OK	OK
subjectList != null	Y	N	N	N
textbookList != null	Y	N	Y	Y
`<option value="<%= sub.getSubjectsNumber()`	X	-	-	X
`<option  value="<%= text.getTextbookNumber() %>"><%= text.getTextbookName() %>`	X	-	X	-

対象となるソースコード

```
<% if (subjectList != null) {
                              for (Subject sub : subjectList) { %>
                                 <option value="<%= sub.getSubjectsNumber() %>"><%= sub.getSubjectsName() %></option>
                         <%   }
                            } %>
<% if (textbookList != null) {
                              for (Textbook text : textbookList) { %>
                                 <option  value="<%= text.getTextbookNumber() %>"><%= text.getTextbookName() %></option>
                         <%   }
                            } %>
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TextDao.java
プログラミング担当	**田畑**
簡単な説明	教科書情報のDao

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04	waribiki-0	waribiki-0	waribiki-0	waribiki-08
テスト担当者	太郎	次郎	三郎	四郎	四郎	四郎	四郎	四郎
結果	OK	OK	OK	OK	OK	OK	OK	OK
PreparedStatement ps = conn.prepareStatement(sql) (登録)	Y	-	-	-	-	-	-	-
(更新)	-	Y	-	-	-	-	-	-
(削除)	-	-	Y	-	-	-	-	-
(教科書番号検索)	-	-	-	Y	-	-	-	-
(キーワード検索)	-	-	-	-	Y	-	-	-
(公開中教科書取得)	-	-	-	-	-	Y	-	-
(管理者用全権取得)	-	-	-	-	-	-	Y	-
Statemnt st = conn.createStatement();	-	-	-	-	-	-	-	Y
ResultSet rs = st.executeQuery(sql)								

-_-

ウィザー

ウィザー

対象となるソースコード

```
 try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, text.getTextbookName());
            ps.setInt(2, text.getStatusId());
            ps.executeUpdate();
        }

 try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, text.getTextbookName());
            ps.setInt(2, text.getStatusId());
            ps.setInt(3, text.getTextbookNumber());
            ps.executeUpdate();
        }

try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, textbookNumber);
            ps.executeUpdate();
        }

try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, textbookNumber);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return map(rs);
                }
            }
        }

try (Statement st = conn.createStatement();
             ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {
                Textbook text = new Textbook();
```

```
                text.setTextbookNumber(rs.getInt("textbook_number"));
                text.setTextbookName(rs.getString("textbook_name"));
                text.setStatusId(rs.getInt("status_id"));
                list.add(text);
            }
        }

try (PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, "%" + keyword + "%");
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                Textbook t = new Textbook();
                t.setTextbookNumber(rs.getInt("textbook_number"));
                t.setTextbookName(rs.getString("textbook_name"));
                textbooks.add(t);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }

try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, Status.TEXTBOOK_AVAILABLE);
            ResultSet rs = ps.executeQuery();

            List<Textbook> list = new ArrayList<>();
            while (rs.next()) list.add(map(rs));
            return list;
        }

try (PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            List<Textbook> list = new ArrayList<>();
            while (rs.next()) list.add(map(rs));
            return list;
        }

try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, Status.TEXTBOOK_UNAVAILABLE);
            ps.setInt(2, textbookNumber);
            return ps.executeUpdate() == 1;
        }
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	SubjectDao.java
プログラミング担当	**田畑**
簡単な説明	科目情報のDao

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04
テスト担当者	太郎	次郎	三郎	四郎
結果	OK	OK	OK	OK
subjectList != null	Y	N	N	N
textbookList != null	Y	N	Y	Y
`<option value="<%= sub.getSubjectsNumber()`	X	-	-	X
`<option  value="<%= text.getTextbookNumber() %>"><%= text.getTextbookName() %>`	X	-	X	-

対象となるソースコード

```
<% if (subjectList != null) {
                              for (Subject sub : subjectList) { %>
                                 <option value="<%= sub.getSubjectsNumber() %>"><%= sub.getSubjectsName() %></option>
                         <%   }
                            } %>
<% if (textbookList != null) {
                              for (Textbook text : textbookList) { %>
                                 <option  value="<%= text.getTextbookNumber() %>"><%= text.getTextbookName() %></option>
                         <%   }
                            } %>
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TextinforegisterServlet.java,Subject.java,Textbook.java
プログラミング担当	**田畑**
簡単な説明	教科書を登録するためのサーブレット

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04	waribiki-0	waribiki-06	
テスト担当者	太郎	次郎	三郎	四郎	四郎	四郎	
結果	OK	OK	OK	OK	OK	OK	
(GET)							
loginUser == null	Y	N	N	N			
response.sendRedirect("LoginServlet"); return;	X	-	-	X	-	-	
[redacted]							
(POST)							
user != null	Y	Y	Y	Y	N	N	N
bookNameStr != null && !bookNameStr.isEmpty()	Y	Y	N	N	Y	Y	N
subjectStr != null && !subjectStr.isEmpty()	Y	N	Y	N	Y	N	Y

pi.setUserId(user.getUserId());
pi.setTextbookNumber(Integer.parseInt(bookNameStr));
pi.setSubjectsNumber(Integer.parseInt(subjectStr));
	X						

pi.setUserId(user.getUserId()); pi.setTextbookNumber(Integer.parseInt(bookNameStr));		X					
pi.setUserId(user.getUserId()); pi.setSubjectsNumber(Integer.parseInt(subjectStr));			X				
pi.setTextbookNumber(Integer.parseInt(bookNameStr)); pi.setSubjectsNumber(Integer.parseInt(subjectStr));					X		
pi.setUserId(user.getUserId());				X			
pi.setTextbookNumber(Integer.parseInt(bookNameStr));						X	
pi.setSubjectsNumber(Integer.parseInt(subjectStr));							X

対象となるソースコード

```
 if (loginUser == null) {
            response.sendRedirect("LoginServlet");
            return;
        }
try {
            TextDao text = new TextDao();
            SubjectDao sub = new SubjectDao();
            List<Textbook> textbook = text.findAll();
            List<Subject> subject = sub.findAll();
            request.setAttribute("Textbook", textbook);
            request.setAttribute("Subject", subject);
            RequestDispatcher dispatcher = request.getRequestDispatcher("WEB-INF/jsp/textinforegister.jsp");
            dispatcher.forward(request, response);
        } catch (SQLException e) {
            // TODO 自動生成された catch ブロック
            e.printStackTrace();
        }
 if (user != null) {
            pi.setUserId(user.getUserId());
        }
if (bookNameStr != null && !bookNameStr.isEmpty()) {
            pi.setTextbookNumber(Integer.parseInt(bookNameStr));
```

ウィザー

ウィザー

```
if (subjectStr != null && !subjectStr.isEmpty()) {
            pi.setSubjectsNumber(Integer.parseInt(subjectStr));
        }
try {
            pro = new ProvisionDao();
            pro.insert(pi);
        } catch (SQLException e) {
            // TODO 自動生成された catch ブロック
            e.printStackTrace();
        }
```

チーム名	**チームC**
試験対象プログラム名	MarkingServlet.java
プログラミング担当	**田畑**
簡単な説明	マーキングを行うサーブレット

テストNo	waribiki-01	waribiki-02
テスト担当者	太郎	次郎
結果	OK	OK
`provisionNumbers != null`	Y	N
`int provisionNumber = Integer.parseInt(pNum);` `logic.registerSuggestion(userId, provisionNumber);`	X	-
	-	-

対象となるソースコード

```
 if (provisionNumbers != null) {
            for (String pNum : provisionNumbers) {
                int provisionNumber = Integer.parseInt(pNum);
                logic.registerSuggestion(userId, provisionNumber);
            }
        }
```

ウィザー

ウィザー

チーム名	**チームC**
試験対象プログラム名	Textinfodelete.java
プログラミング担当	**田畑**
簡単な説明	教科書情報削除を行う画面

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04
テスト担当者	太郎	次郎	三郎	四郎
結果	OK	OK	OK	OK
`provision_id == null`	Y	N	N	Y

ProvisionDao prodao = new ProvisionDao();
SuggestedInfoDao sugdao = new SuggestedInfoDao();
prodao.delete(pronumber);
sugdao.deleteByprovider_id(pronumber);
	Y	N	Y	N

`RequestDispatcher dispatcher = request.getRequestDispatcher("GiveHomeServlet");`	X	-	-	X
e.printStackTrace();	-	X	-	-

対象となるソースコード

```
if (provision_id == null) {
            RequestDispatcher dispatcher = request.getRequestDispatcher("GiveHomeServlet");
            dispatcher.forward(request, response);
        }
try {
            ProvisionDao prodao = new ProvisionDao();
            SuggestedInfoDao sugdao = new SuggestedInfoDao();
            prodao.delete(pronumber);
            sugdao.deleteByprovider_id(pronumber);
        } catch (SQLException e) {
            // TODO 自動生成された catch ブロック
            e.printStackTrace();


        }
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TextsSearchServlet.java,SearchResult.java
プログラミング担当	**田畑**
簡単な説明	教科書の一覧を出すためのプログラム

テストNo	waribiki-01	waribiki-02	waribiki-03
テスト担当者	太郎	次郎	三郎
結果	OK	OK	OK
(GET)			
`result = logic.execute(null);`	Y	Y	N
result == null	Y	N	
`e.printStackTrace();`	-	-	X
`result = new SearchResult();`	X	-	-
[redacted]			
(POST)			
`result = logic.execute(keyword);`	Y	Y	N
`result == null`	Y	N	Y
`e.printStackTrace();`	-	-	X
`result = new SearchResult();`	X	-	-

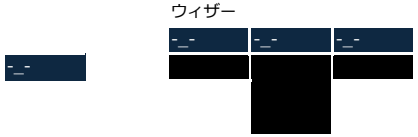
対象となるソースコード

```
 try {
            result = logic.execute(null); // 全件取得
        } catch (SQLException e) {
            e.printStackTrace();
        }

        // result が null の場合は空の SearchResult を作る
        if (result == null) {
            result = new SearchResult();
        }

try {
            result = logic.execute(keyword); // キーワード検索
        } catch (SQLException e) {
            e.printStackTrace();
        }

        // result が null の場合は空の SearchResult を作る
        if (result == null) {
            result = new SearchResult();
        }
```



デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	TextSearchLogic.java
プログラミング担当	**名前**
簡単な説明	教科書検索を行うDAOに接続する

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04	waribiki-0	waribiki-04
テスト担当者	太郎	次郎	三郎	四郎	四郎	四郎
結果	OK	OK	OK	OK	OK	OK
`keyword == null || keyword.isBlank()`	Y	Y	Y	N	N	N
`provisions == null`	Y	Y	N	Y	Y	N
`!subjectMap.containsKey(subjectId)`	Y	N	N	Y	N	N
`s != null`	Y	N	N	Y	N	N
`t != null && !textbooks.contains(t)`	Y	Y	Y	Y	Y	Y

`provisions = pDao.findAll();`
provisions = new ArrayList<>()
Subject s = sDao.findById(subjectId);
subjectMap.put(subjectId, s);
textbooks.add(t);
	X	-	-	-	-	-

`provisions = pDao.findAll();`
provisions = new ArrayList<>()
textbooks.add(t);
	-	X	-	-	-	-

`provisions = pDao.findAll();` textbooks.add(t);	-	-	X	-	-	-

`provisions = pDao.searchByKeyword(keyword);`
provisions = new ArrayList<>()
Subject s = sDao.findById(subjectId);
subjectMap.put(subjectId, s);
textbooks.add(t);
	-	-	-	X	-	-

provisions = pDao.searchByKeyword(keyword);
provisions = new ArrayList<>()
textbooks.add(t);
	-	-	-	-	X	-

provisions = pDao.searchByKeyword(keyword); textbooks.add(t);	-	-	-	-	-	X

対象となるソースコード

```
 if (keyword == null || keyword.isBlank()) {
            provisions = pDao.findAll();
        } else {
            provisions = pDao.searchByKeyword(keyword);
        }
        if (provisions == null) provisions = new ArrayList<>();

if (!subjectMap.containsKey(subjectId)) {
                Subject s = sDao.findById(subjectId);
                if (s != null) subjectMap.put(subjectId, s);
            }

if (t != null && !textbooks.contains(t)) textbooks.add(t);
```

チーム名	**チームC**
試験対象プログラム名	MarkingLogic.java
プログラミング担当	**田畑**
簡単な説明	マーキング処理を行うDAOに接続する

テストNo	waribiki-01	waribiki-02	waribiki-03
テスト担当者	太郎	次郎	次郎
結果	OK	OK	OK

`SuggestedInfoDao dao = new SuggestedInfoDao();`
`SuggestedInfo su = new SuggestedInfo();`
`su.setUserId(userId);`
`su.setProvisionNumber(provisionNumber);`
`int result = dao.insert(su);`
	Y	Y	N

`dao.exists(userId, provisionNumber)`	Y	N	N
`return false;`	X	-	-
return result > 0;	-	X	-
e.printStackTrace(); return false;	-	-	X

対象となるソースコード

```
try {
            SuggestedInfoDao dao = new SuggestedInfoDao();

            // ★ 重複チェック
            if (dao.exists(userId, provisionNumber)) {
                // すでに登録済み
                return false;
            }

            SuggestedInfo su = new SuggestedInfo();
            su.setUserId(userId);
            su.setProvisionNumber(provisionNumber);

            int result = dao.insert(su);

            return result > 0;  // 1行追加されたら成功

        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
```

ウィザー

ウィザー

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	MypageServlet.java
プログラミング担当	**担当者の名前**
簡単な説明	マイページ表示処理

テストNo	MypageServlet01	MypageServlet02	MypageServlet03	MypageServlet04
テスト担当者	野中	野中	野中	野中
結果				
session != null（セッションあり）	N	Y	Y	Y
loginUser == null（loginUserチェック）	N	N	Y	Y
dao = new UserDao();（DAO生成）	-	-	N	Y
identityImagePath = dao.findIdentityImagePathByUserId(loginUser.getUserId());（i	-	-	-	Y

ウィザー

ウィザー

対象となるソースコード

```
HttpSession session = request.getSession(false);
User loginUser = (session != null)
? (User) session.getAttribute("loginUser")
: null;

if (loginUser == null) {
response.sendRedirect(request.getContextPath() + "/user_Login.jsp");
return;
}

try {
dao = new UserDao();
} catch (SQLException e) {
throw new ServletException("DB接続に失敗しました", e);
}

try {
identityImagePath = dao.findIdentityImagePathByUserId(loginUser.getUserId());
} catch (SQLException e) {
throw new ServletException("本人確認画像の取得に失敗しました", e);
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	SuggestedListServlet
プログラミング担当	**名前 松田**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04	waribiki-05
テスト担当者	松田	松田	松田	松田	松田
結果	OK	OK	OK	OK	OK
session == null)	Y	N	N	N	N
loginUser == null	-	Y	N	N	N
DBからの取得	-	-	Y	N	-
SQLExceptionの発生	-	-	N	N	Y
response.sendRedirect(ı	X	X	-	-	-
response.sendRedirect(ı	-	-	-	-	X
result = logic.execute(u	-	-	X	X	-
	-	-	X	X	-

対象となるソースコード

```
if (session == null) {
        response.sendRedirect(request.getContextPath() + "/LoginServlet");
        return;
    }
if (loginUser == null) {
        response.sendRedirect(request.getContextPath() + "/LoginServlet");
        return;
```

```
        }
}
try {
            result = logic.execute(userId);
        } catch (SQLException e) {
            throw new ServletException(e);
        }
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	DbConnect.java
プログラミング担当	**井上綾人**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	DbConnect-01	DbConnect-02
テスト担当者	井上綾人	井上
結果	OK	OK
データベース起動時	Y	N
Class.forName("org.mariadb.jdbc.Driver");	X	-
e.printStackTrace();	-	X
throw new RuntimeException("MariaDB JDBC Driver が見つかりません", e);		

対象となるソースコード

if (会員カード＝なし) {

```
try {
        // MariaDB JDBC ドライバをロード
        Class.forName("org.mariadb.jdbc.Driver");
    } catch (ClassNotFoundException e) {
        e.printStackTrace();
        throw new RuntimeException("MariaDB JDBC Driver が見つかりません", e);
    }
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminListRegistrationLogic
プログラミング担当	**和合由樹**
簡単な説明	業務ロジックを実行し、DAOやモデルを組み合わせてユースケース処理を行います。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								
try { int result = userDao.insert(user);	Y	N								
対象となるソースコード□										

return result > 0; // 登録成功した場合は true を返す
e.printStackTrace();
	X									

return false; // エラーが発生した場合は false を返す		X								

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminListConformLogic
プログラミング担当	**和合由樹**
簡単な説明	業務ロジックを実行し、DAOやモデルを組み合わせてユースケース処理を行います。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								

対象となるソースコード□

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminListRetireLogic
プログラミング担当	**和合由樹**
簡単な説明	業務ロジックを実行し、DAOやモデルを組み合わせてユースケース処理を行います。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								

対象となるソースコード□

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminListLogic
プログラミング担当	**和合由樹**
簡単な説明	業務ロジックを実行し、DAOやモデルを組み合わせてユースケース処理を行います。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								

対象となるソースコード□

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminListDeleteLogic
プログラミング担当	**和合由樹**
簡単な説明	業務ロジックを実行し、DAOやモデルを組み合わせてユースケース処理を行います。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								

対象となるソースコード□

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminSearchListLogic
プログラミング担当	**和合由樹**
簡単な説明	業務ロジックを実行し、DAOやモデルを組み合わせてユースケース処理を行います。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								

対象となるソースコード□

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	administrator_registration.jsp
プログラミング担当	**和合由樹**
簡単な説明	処理概要: 画面表示とユーザー操作の受付を行い、必要に応じてJavaScriptで表示制御します。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								
if (!inputElement || !outputElement)	Y	N								
if (isPassword)			Y	N						
			-				-	-	-	-
return;	X	-	-	-		-	-	-	-	-
displayValue = displayValue.length > 0 ? '*'.repeat(displayValue.length) : '';			X							
					-					
						-	-			
							-		-	
									-	

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	administrator_List.jsp
プログラミング担当	**和合由樹**
簡単な説明	処理概要: 画面表示とユーザー操作の受付を行い、必要に応じてJavaScriptで表示制御します。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								
if (selectedAdmin)	Y	N								
if (list != null)			Y	N						
for (User admin : list)			-		Y	N	-	-	-	-
if ("admin".equals(admin.getRole()))							Y	N		
updateButton.disabled = false; deleteButton.disabled = false;	X	-	-	-		-	-	-	-	-
updateButton.disabled = true; deleteButton.disabled = true;		X								
for (User admin : list) {			X		-					
if ("admin".equals(admin.getRole())) {					X	-	-			
<tr> <td><input type="radio" name="selectedAdmin" value="<%= admin.getUserId() %>" onclick="toggleActionButtons()"></td> <td><%= admin.getFullName() %></td> <td><%= admin.getUserId() %></td> <td><%= admin.getEmailAddress() %></td> <td><%= admin.getPhoneNumber() %></td> </tr>							X		-	
									-	

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	userApprove.jsp
プログラミング担当	**和合由樹**
簡単な説明	処理概要: 画面表示とユーザー操作の受付を行い、必要に応じてJavaScriptで表示制御します。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	administrator_Delete.jsp
プログラミング担当	**和合由樹**
簡単な説明	処理概要: 画面表示とユーザー操作の受付を行い、必要に応じてJavaScriptで表示制御します。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	administrator_updated.jsp
プログラミング担当	**和合由樹**
簡単な説明	処理概要: 画面表示とユーザー操作の受付を行い、必要に応じてJavaScriptで表示制御します。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								
if (!inputElement || !outputElement)	Y	N								
if (isPassword)			Y	N						
			-				-	-	-	-
return;	X	-	-	-		-	-	-	-	-
displayValue = displayValue.length > 0 ? '*'.repeat(displayValue.length) : '';			X							
					-					
						-	-			
							-		-	
									-	

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminUpdateServlet
プログラミング担当	**和合由樹**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値を解釈して業務ロジックを呼び出し、遷移先を決定します。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								
try {userLogic = new AdminListRegistrationLogic();}	Y	N								
if (!dir.exists())			Y	N						
if (registrationSuccess)			-		Y	N	-	-	-	-
try {										
if (userLogic != null) {										
userLogic = null; // Logicのクリーンアップ（自動で不要なリソースが解放されることが多い）							Y	N		
}										
if (userLogic != null)									Y	N
userLogic = new AdminListRegistrationLogic();	X	-	-	-		-	-	-	-	-
dir.mkdirs();			X							
response.sendRedirect("AdminServlet");					X					
response.sendRedirect("WEB-INF/jsp/administrator_registration.jsp");						X	-			
if (userLogic != null)							X		-	
userLogic = null; // Logicのクリーンアップ（自動で不要なリソースが解放されることが多い）									X	

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminRegistrationServlet
プログラミング担当	**和合由樹**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値を解釈して業務ロジックを呼び出し、遷移先を決定します。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10
テスト担当者	太郎	次郎								
結果	OK	OK								
try {userLogic = new AdminListRegistrationLogic();}	Y	N								
if (!dir.exists())			Y	N						
if (registrationSuccess)			-		Y	N	-	-	-	-
try {										
if (userLogic != null) {										
userLogic = null; // Logicのクリーンアップ（自動で不要なリソースが解放されることが多い）							Y	N		
}										
if (userLogic != null)									Y	N
userLogic = new AdminListRegistrationLogic();	X	-	-	-		-	-	-	-	-
dir.mkdirs();			X							
response.sendRedirect("AdminServlet");					X					
response.sendRedirect("WEB-INF/jsp/administrator_registration.jsp");						X	-			
if (userLogic != null)							X		-	
userLogic = null; // Logicのクリーンアップ（自動で不要なリソースが解放されることが多い）									X	

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminActionServlet.java
プログラミング担当	**和合由樹**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値を解釈して業務ロジックを呼び出し、遷移先を決定します。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06	Admin-07	Admin-08	Admin-09	Admin-10	Admin-11	Admin-12
テスト担当者	太郎	次郎										
結果	OK	OK										
if (selected == null && !action.equals("register"))	Y	N										
if (action.equals("update"))			Y	N								
try {												
AdminInfo = AdminSearchListLogic.getAdminById(userId);			-		Y	N	-	-	-	-	-	-
if (action.equals("delete"))							Y	N				
try {												
AdminInfo = AdminSearchListLogic.getAdminById(userId);									Y	N	-	-
if (action.equals("register"))											Y	N
throw new ServletException("管理者が選択されていません")	X	-	-	-		-	-	-	-	-	-	-
int userId = Integer.parseInt(selected);												
AdminSearchListLogic AdminSearchListLogic = new AdminSearchListLogic();												
User adminInfo =null;System.out.print(adminInfo);												
request.setAttribute("admin", adminInfo);			X									
RequestDispatcher dispatcher = request.getRequestDispatcher("WEB-INF/jsp/administrator_updated.jsp");												
dispatcher.forward(request, response);												
catch (SQLException e) {												
// TODO 自動生成された catch ブロック					X							
e.printStackTrace();												
int userId = Integer.parseInt(selected);												
AdminSearchListLogic AdminSearchListLogic = new AdminSearchListLogic();												
User adminInfo = null;System.out.print(adminInfo);												
request.setAttribute("admin", adminInfo);							X					
RequestDispatcher dispatcher = request.getRequestDispatcher("WEB-INF/jsp/administrator_Delete.jsp");												
dispatcher.forward(request, response);												

```
catch (SQLException e) {
// TODO 自動生成された catch ブロック
e.printStackTrace();
RequestDispatcher dispatcher =
request.getRequestDispatcher("WEB-
INF/jsp/administrator_registration.jsp");
    dispatcher.forward(request, response);
```

	X	
		X

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminHomeServlet.java
プログラミング担当	**和合由樹**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値を解釈して業務ロジックを呼び出し、遷移先を決定します。

テストNo	Admin_01	Admin-02	Admin-03	Admin-04	Admin-05	Admin-06
テスト担当者	太郎	次郎				
結果	OK	OK				
if (session == null)	Y	N				
if (loginUser == null)	-	-	Y	N		
if (!loginUser.isAdmin())	-	-			Y	N
response.sendRedirect(request.getContextPath() + "/User_LoginServlet"); return;	-	X	-	X		X

対象となるソースコード

```
if (session == null) {
            response.sendRedirect(request.getContextPath()
+ "/User_LoginServlet");
            return;
        }

        User loginUser = (User)
session.getAttribute("loginUser");
        if (loginUser == null) {
            response.sendRedirect(request.getContextPath()
+ "/User_LoginServlet");
            return;
        }

        if (!loginUser.isAdmin()) {
            session.setAttribute("errorMsg", "管理者権限があり
ません。");
            response.sendRedirect(request.getContextPath()
+ "/User_LoginServlet");
            return;
        }
        // 管理者ホームJSPへ遷移
```

ウィザー

ウィザー

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	AdminServlet.java
プログラミング担当	**和合由樹**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値を解釈して業務ロジックを呼び出し、遷移先を決定します。

テストNo	Admin_01	Admin-02
テスト担当者	太郎	次郎
結果	OK	OK
adminInfoList = adminListLogic.execute();	Y	N
e.printStackTrace(); response.sendError (HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "データベースエラー"); return;	-	X

対象となるソースコード

```
try{
adminInfoList = adminListLogic.execute();
}catch (SQLException e){
e.printStackTrace();
response.sendError
(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
"データベースエラー");
return;
}
```

ウィザー

ウィザー

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	GiveHomeServlet.java
プログラミング担当	**又間**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値を解釈して業務ロジックを呼び出し、遷移先を決定します。

テストNo	GiveHome-01	GiveHome-02	GiveHome-03
テスト担当者	太郎	次郎	三郎
結果	OK	OK	OK
loginUser == null	Y	N	N
logic.getMyProvisionList()	N	Y	N
request.getRequestDispatcher("WEB-INF/jsp/Home.jsp").forw	X	-	-
throw new ServletException(e)	-	X	-
request.setAttribute("provisionList", provisionList)	-	-	X
dispatcher.forward(request, response)	-	-	X

対象となるソースコード

```
if (loginUser == null) {
    response.sendRedirect("LoginServlet");
    return;
}

List<ProvisionView> provisionList =
        logic.getMyProvisionList(loginUser.getUserId());

} catch (Exception e) {
    throw new ServletException(e);
}

request.setAttribute("provisionList", provisionList);
```

```
dispatcher.forward(request, response);
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	GiveHomeLogic.java
プログラミング担当	**又間**
簡単な説明	業務ロジックを実行し、DAOやモデルを組み合わせてユースケース処理を行います。

テストNo	LoginLogic-01	LoginLogic-02
テスト担当者	太郎	次郎
結果	OK	OK
dao.findByUserId(userId)	Y	N
return List<ProvisionView>	X	-

対象となるソースコード

```
dao.findByUserId(userId);

return dao.findByUserId(userId);
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	givehomebtn.jsp
プログラミング担当	**又間**
簡単な説明	JavaScriptで表示要素の表示/非表示や状態切替を行う画面です。

テストNo	givehomebtn-01	givehomebtn-02	givehomebtn-03	givehomebtn-04	givehomebtn-05
テスト担当者	太郎	次郎	太郎	次郎	次郎
結果	OK	OK	OK	OK	OK
path.includes('transactionHistory')	Y	N	N	N	N
path.includes('provisionList')	N	Y	Y	N	N
path.includes('SuggestedListServlet')	N	N	N	Y	N
window.addEventListener('DOMContentLoaded')	Y	Y	Y	Y	Y
document.getElementById('btn-history').classList.add('active')	X	-	-	-	-
document.getElementById('btn-provision').classList.add('active')	-	X	X	-	-
document.getElementById('SuggestedList').classList.add('active')	-	-	-	X	-

対象となるソースコード

```
window.addEventListener('DOMContentLoaded', () => {

if (path.includes('transactionHistory'))

document.getElementById('btn-history').classList.add('active');

else if (path.includes('provisionList'))
```

```
document.getElementById('btn-provision').classList.add('active');

else if (path.includes('provisionList'))

document.getElementById('btn-provision').classList.add('active');

else if (path.includes('SuggestedListServlet'))

document.getElementById('SuggestedList').classList.add('active');
});
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	giveHome.jsp
プログラミング担当	**又間**
簡単な説明	画面表示とユーザー操作の受付を行い、必要に応じてJavaScriptで表示制御します。

テストNo	giveHome-01	giveHome-02	giveHome-03	giveHome-04
テスト担当者	太郎	次郎	太郎	次郎
結果	OK	OK	OK	OK
loginUser != null	Y	N	Y	Y
list == null || list.isEmpty()	N	Y	N	N
ProvisionView p : list	N	N	Y	Y
querySelector('input[name="selectedId"]:checked')	N	N	Y	N
<p class="login-user-name">	X	-	X	X
<td style="text-align: center; padding: 20px;">現在、提供中の教科	-	X	-	-
<tr>～	-	-	X	X
<button type="submit">削除</button>	-	-	X	X

対象となるソースコード

```
if (loginUser != null)

if (list == null || list.isEmpty())

for (ProvisionView p : list)

const isChecked = document.querySelector('input[name="selectedId"]:checked') !== null;
```

```
<p class="login-user-name">

<td style="text-align: center; padding: 20px;">現在、提供中の教科書はありません。</td>

<tr>
    <td>
        <div class="textbook-item">

<button type="submit">削除</button>
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	LoginLogic.java
プログラミング担当	**又間**
簡単な説明	業務ロジックを実行し、DAOやモデルを組み合わせてユースケース処理を行います。

テストNo	LoginLogic-01	LoginLogic-02
テスト担当者	太郎	次郎
結果	OK	OK
userDao.userLogin(name, password)	Y	N
return user	X	-

対象となるソースコード

```
userDao.userLogin(name, password);

return user;
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	HomeServlet.java
プログラミング担当	**又間**
簡単な説明	ServletとしてHTTPリクエストを受け取り、入力値を解釈して業務ロジックを呼び出し、遷移先を決定します。

テストNo	Home-01	Home-02
テスト担当者	太郎	次郎
結果	OK	OK
request.getRequestDispatcher("WEB-INF/jsp/Home.jsp")	Y	N
.forward(request, response);	X	-

対象となるソースコード

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    request.getRequestDispatcher("WEB-INF/jsp/Home.jsp").forward(request, response);
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	SuggestedListLogic
プログラミング担当	**名前　松田**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04
テスト担当者	松田	松田	松田	松田
結果	OK	OK	OK	OK
userid の状態	Y	Y	N	Y
DAOの実行結果	Y	N	Y	N
DBの接続状態	Y	Y	Y	N
findByRequesterUserId 呼出	X	X	X	X
Listオブジェクトを返却	X	X	X	-
SQLException をスロー	-	-	-	X

対象となるソースコード

```
SuggestedTextbookDao SuggestedTextbookDao = new SuggestedTextbookDao();
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	ProvisionListServle
プログラミング担当	**名前 井上**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	ProvisionList-01	ProvisionList-02	ProvisionList-03
テスト担当者	井上	井上	井上
結果	OK	OK	OK
loginUser == null	Y	N	Y
DBからの取得	Y	-	N
SQLExceptionの発生	N	-	Y
response.sendRedirect(request.getContextPath() + "/login"); ,	-	X	-
List<ProvisionView> provisionList = logic.execute(loginUser.getUserId());	X	-	X
throw new ServletException(e);	-	-	X

対象となるソースコード

```
if (loginUser == null) {
        response.sendRedirect(request.getContextPath() +
"/login"); // パスは環境に合わせて調整
        return;
    }
 try {
        ProvisionListLogic logic = new ProvisionListLogic();
        List<ProvisionView> provisionList =
logic.execute(loginUser.getUserId());

        // JSPに渡す名前を "provisionList" で確定
        request.setAttribute("provisionList", provisionList);

        // デバッグ用：中身が空でないか確認
        //System.out.println("取得件数: " + (provisionList !=
null ? provisionList.size() : "null"));

        request.getRequestDispatcher("WEB-
INF/jsp/provisionList.jsp").forward(request, response);

    } catch (SQLException e) {
        throw new ServletException(e);
    }
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	Profile_UpdateTransitionServlet.java
プログラミング担当	**名前**
簡単な説明	プロフィール更新画面への遷移処理

テストNo	Profile_UpdateTransitionServlet-1	Profile_UpdateTransitionServleti-02
テスト担当者	野中	野中
結果	OK	OK
loginUser == null	Y	N

ウィザー

ウィザー

対象となるソースコード

```
if (loginUser == null) {
response.sendRedirect("WEB-INF/jsp/user_Login.jsp");
return;
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	Provider.jsp
プログラミング担当	**名前　松田**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	Provider.jsp-01	Provider.jsp-02
テスト担当者	松田	松田
結果	OK	OK
ラジオボタンの選択状態	Y	N
onchange イベントの発生	-	Y
送信ボタン (sendBtn)	Y	Y
取引完了ボタン (completeBtn)	Y	Y
キャンセルボタン (cancelBtn)	Y	Y

対象となるソースコード

```
document.getElementById("sendBtn").disabled = false;
document.getElementById("completeBtn").disabled = false;
document.getElementById("cancelBtn").disabled = false;
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	Profile_UpdateServlet.java
プログラミング担当	**名前**
簡単な説明	プロフィール更新処理

条件 / テストNo	**Profile_UpdateS**	**Profile_UpdateS**	**Profile_UpdateS**	**Profile_UpdateServlet-04**
テスト担当者	野中	野中	野中	野中
結果	OK	OK	OK	OK
条件	**waribiki-01**	**waribiki-02**	**waribiki-03**	**waribiki-04**
idStr == null || idStr.isEmpty()	N	N	N	N
oldUser == null（DB取得）	N	N	N	Y
newUsersPhoto != null（本人画像）	Y	N	Y	N
newIdentityImage != null（証明画像）	Y	Y	N	N

対象となるソースコード

```
if (idStr == null || idStr.isEmpty()) {
throw new RuntimeException("managerId が送信されていません。");
}

if (oldUser == null) {
throw new RuntimeException("該当ユーザーが存在しません (userId=" + userId + ")");
}

String usersPhoto = (newUsersPhoto != null) ? newUsersPhoto : oldUser.getUsersPhoto();

if (newIdentityImage != null) {
// user_identity_verification テーブルへ保存（承認待ち）
dao.upsertIdentityImage(userId, newIdentityImage);
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	ProvisionListLogic
プログラミング担当	**名前**
簡単な説明	データベースから必要な提供者の情報を参照する

テストNo	ProvisionList-01	ProvisionList-02
テスト担当者	太郎	次郎
結果	OK	OK
データを取得	Y	N
return dao.findProviderinfoByUserId(userId);	X	-

対象となるソースコード

```
public class ProvisionListLogic {
    public List<ProvisionView> execute(int userId) throws
SQLException {
        ProvisionDao dao = new ProvisionDao();
        // 新しく作成した提供者情報取得メソッドを呼び出す
        return dao.findProviderinfoByUserId(userId);
    }
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	SuggestedTextbookDao
プログラミング担当	**名前　松田**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	SuggestedTextbookDao-01	SuggestedTextbookDao-02	SuggestedTextbookDao-03	SuggestedTextbookDao-04
テスト担当者	松田	松田	松田	松田
結果	OK	OK	OK	OK
セッション・ログインユーザーあり	Y	Y	N	Y
DBに対象データが存在する	Y	N	-	-
SQL実行時の例外(SQLException)	N	N	-	Y
findByRequesterUserId 実行	X	X	-	X
結果（List型）の返却	X	-	-	-
ログイン画面へリダイレクト	-	-	X	-
ServletExceptionを送出	-	-	-	X

対象となるソースコード

```
if (session == null || loginUser == null) {
    response.sendRedirect(request.getContextPath() +
"/LoginServlet");
    return;
}
try {
    // DAOの呼び出し
    SuggestedTextbookDao dao = new
SuggestedTextbookDao();
    List<SuggestedTextbookView> list =
dao.findByRequesterUserId(userId);

    // Case-01: list.size() > 0
    // Case-02: list.size() == 0 (正常終了だが表示データなし)
} catch (SQLException e) {
    // Case-04: DB接続失敗やSQLエラー
    throw new ServletException(e);
}
```

ウィザー

ウィザー

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	Profile_DeleteServlet.java
プログラミング担当	**名前**
簡単な説明	プロフィールの削除

テストNo	**Profile_DeleteSe**	**Profile_DeleteSe**	**Profile_DeleteSe**	**Profile_DeleteServlet-04**
テスト担当者	太郎	次郎	三郎	四郎
結果	OK	OK	OK	NG
session == null	N	Y	N	N
loginUser == null	N	-	Y	N
result = logic.delete(loginUser.getUserId());	Y	-	-	N

対象となるソースコード

```
if (session == null) {
response.sendRedirect("login.jsp");
return;
}

if (loginUser == null) {
response.sendRedirect("login.jsp");
return;
}

try {
result = logic.delete(loginUser.getUserId());
} catch (SQLException e) {
e.printStackTrace();
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	Providerconpservlet
プログラミング担当	**名前　井上**
簡単な説明	取引を行う際に使用されるサーブレット

テストNo	Providerconp-01	Providerconp-02	Providerconp-03	Providerconp-04
テスト担当者	太郎	次郎	三郎	四郎
結果	OK	OK	OK	OK
tradeSelect パラメータが存在する	Y	N	Y	Y
tradeSelect が数値に変換可能	Y	-	N	Y
DB接続が成功する	Y	-	-	N
completeTrade() が true を返す	Y	-	-	-
TransactionProviderservlet へリダイレクト	X	-	-	-
res.sendError(HttpServletResponse.SC_BAD_REQUEST, "取引が選択されていません"); return;	-	X	-	-
ServletException をスロー（不正な取引番号）	-	-	X	-
ServletException をスロー（SQLException）	-	-	-	X

対象となるソースコード

```
if (tradeSelect == null) {

res.sendError(HttpServletResponse.SC_BAD_REQUEST, "取引
が選択されていません");
        return;
    }
try {
        tradingNumber = Integer.parseInt(tradeSelect);
    } catch (NumberFormatException e) {
        throw new ServletException("不正な取引番号です",
e);
    }
```

```
try (Connection conn = DbConnect.getConnection()) {

            TextTradeDao tradeDao = new TextTradeDao();

            boolean updated =
tradeDao.completeTrade(tradingNumber);

            if (!updated) {

res.sendError(HttpServletResponse.SC_CONFLICT, "取引を完
了できませんでした");
                return;
            }

            // 完了後は一覧に戻す
            res.sendRedirect("TransactionProviderservlet");

        } catch (SQLException e) {
            throw new ServletException(e);
        }
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	SuggestedInfoDao
プログラミング担当	**名前**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	SuggestedInfoDao-01	SuggestedInfoDao-02	SuggestedInfoDao-03	SuggestedInfoDao-04	SuggestedInfoDao-05
テスト担当者	松田	松田	松田	松田	松田
結果	OK	OK	OK	OK	OK
セッション・ログインあり	Y	Y	Y	Y	Y
DBに対象データが存在する	N	Y	N	N	-
SQL実行時に例外が発生した	N	N	N	N	Y
insert (新規登録) 実行	X	-	-	-	X
exists (存在確認) が True	-	X	-	-	-
delete (削除) 実行結果が 0件	-	-	X	-	-
findById (検索) 結果が null	-	-	-	X	-
ServletExceptionを送出	-	-	-	-	X

ウィザー

ウィザー

対象となるソースコード

```
public int insert(SuggestedInfo si) throws SQLException { String sql = "INSERT INTO suggested_info (user_id, provision_number) VALUES (?, ?)"; try (PreparedStatement pst = conn.prepareStatement(sql, PreparedStatement.RETURN_GENERATED_KEYS)) { pst.setInt(1, si.getUserId()); pst.setInt(2, si.getProvisionNumber()); pst.executeUpdate(); // ここでDBへ書き込み try (ResultSet rs = pst.getGeneratedKeys()) { if(rs.next()) return rs.getInt(1); // 成功：新しいIDを返す } } return -1; }
public boolean exists(int userId, int provisionNumber) throws SQLException { String sql = "SELECT 1 FROM suggested_info WHERE user_id = ? AND provision_number = ?"; try (PreparedStatement ps = conn.prepareStatement(sql)) { ps.setInt(1, userId); ps.setInt(2, provisionNumber); try (ResultSet rs = ps.executeQuery()) { return rs.next(); // データがあれば true (X) } } }
public boolean delete(int suggestedNumber) throws SQLException { String sql = "DELETE FROM suggested_info WHERE suggested_number=?"; try (PreparedStatement pst = conn.prepareStatement(sql)) { pst.setInt(1, suggestedNumber); // executeUpdate() が 0 を返すと、結果は false (X) になる return pst.executeUpdate() > 0; } }
public SuggestedInfo findById(int suggestedNumber) throws SQLException { String sql = "SELECT * FROM suggested_info WHERE suggested_number=?"; try (PreparedStatement pst = conn.prepareStatement(sql)) { pst.setInt(1, suggestedNumber); try (ResultSet rs = pst.executeQuery()) { if(rs.next()) return mapToEntity(rs); } } return null; // データがない場合は null (X) を返す }
```

デシジョンテーブル

チーム名	**チームC**	
試験対象プログラム名	MypageLogic.java	
プログラミング担当	**名前**	
簡単な説明		
テストNo	**MypageLogic-01**	**MypageLogic-02**
テスト担当者	太郎	次郎
結果	OK	OK
DAO処理で例外発生	N	Y
identityImagePath取得成功	Y	-

対象となるソースコード

```
try {
        UserDao userDao = new UserDao();

        // 証明画像パス取得
        String identityImagePath =

userDao.findIdentityImagePathByUserId(loginUser.getUserId
());

        request.setAttribute("identityImagePath",
identityImagePath);

      } catch (Exception e) {
        e.printStackTrace();
        request.setAttribute("identityImagePath", null);
      }
    }
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	ProfileDeleteLogic.java
プログラミング担当	**名前**
簡単な説明	

テストNo	**ProfileDeleteLog**	**ProfileDeleteLogic-02**
テスト担当者	野中	野中
結果	OK	NG
DAO処理で例外発生（deleteUser）	N	Y

対象となるソースコード

```
public User findById(int userId) throws SQLException {
        return dao.findById(userId);
    }

    // 管理者削除
    public boolean delete(int userId) throws SQLException {

        dao.deleteUser(userId);

    }


    // 全件取得
    public List<User> findAll() throws SQLException {
        return dao.findAll();
    }
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	ProvisionDao
プログラミング担当	**名前**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	insert-01	insert-02	insert-03	update/delete-04	update/delete-05	update/delete-06	waribiki-0	waribiki-0	waribiki-09
テスト担当者	井上	次郎	三郎	四郎	四郎	四郎	四郎	四郎	四郎
結果	OK	OK	OK	OK	OK	OK	OK	OK	OK
SQL実行が成功する（例外なし）	Y	Y	N	Y	Y	N	Y	Y	N
対象レコードが存在する	-	-	-	Y	N	-	Y	-	-
対象レコードが存在しない	-	-	-	N	Y	-	N	-	-
自動採番キーが取得できる（insert	Y	N	-	-	-	-	-	-	-
自動採番キーが取得できない（insert	N	Y	-	-	-	-	-	-	-
検索結果が1件以上ある（一覧系）	-	-	-	-	-	-	-	Y	-
検索結果が0件（一覧系）	-	-	-	-	-	-	-	N	-
自動採番キーの数値を取得(結果)	X	-	-	-	-	-	-	X	X
-1を返す	-	X	-	-	-	-	-	-	-
Trueを返す	-	-	-	X	-	-	-	-	-
Falseを返す	-	-	-	-	X	-	-	-	-
エンティティ(Provision)	-	-	-	-	-	-	-	-	-
nullを返す	-	-	-	-	-	-	-	-	-
1件以上のリスト(List)を返す	-	-	-	-	-	-	-	-	-
空のリストを返す	-	-	-	-	-	-	-	-	-
例外発生:SQLException	-	-	X	-	-	X	-	-	-

対象となるソースコード

```
if (会員カード＝なし) {
    料金＝3,000
} else {
    if (学生証＝あり) {
        料金＝2,500
    } else {
        料金＝2,800
    }
}
```

ウィザー

ウィザー

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	User.java
プログラミング担当	**名前**
簡単な説明	usersテーブルに対応するエンティティクラス ＊管理者・一般ユーザなどを含む全ユーザ情報を保持する。

テストNo				
テスト担当者				
結果				
	-	-	-	-
	-	-	-	-
	-	-	-	-
	-	-	-	-
	-	-	-	-
	-	-	-	-

対象となるソースコード

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	UserDao.java
プログラミング担当	**名前**
簡単な説明	DAOとしてデータベースに対する検索・登録・更新・削除処理

テストNo	UserDao-01	UserDao-02	UserDao-03	UserDao-04	UserDao-05	UserDao-06	UserDao-07	UserDao-08
テスト担当者	野中	野中	野中	野中	野中	野中	野中	野中
結果	OK	OK	OK	OK	OK	OK	OK	OK
INSERT	Y	-	-	-	-	-	-	-
UPDATE	-	Y	-	-	-	-	-	-
DELETE	-	-	Y	-	-	-	-	-
全件取得	-	-	-	Y	-	-	-	-
ログイン認証	-	-	-	-	Y	-	-	-
１件取得	-	-	-	-	-	Y	-	-
本人確認用 証明画像パス取得（最新1件）	-	-	-	-	-	-	Y	-
本人確認画像 登録 or 更新	-	-	-	-	-	-	-	Y

対象となるソースコード

```
try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, user.getPassword());
        ps.setString(2, user.getRole());
        ps.setString(3, user.getFullName());
        ps.setString(4, user.getEmailAddress());
        ps.setString(5, user.getPhoneNumber());
        ps.setString(6, user.getUsersPhoto());
        return ps.executeUpdate();
    }


try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, user.getPassword());
        ps.setString(2, user.getRole());
        ps.setString(3, user.getFullName());
        ps.setString(4, user.getEmailAddress());
        ps.setString(5, user.getPhoneNumber());
        ps.setString(6, user.getUsersPhoto());
        ps.setInt(7, user.getStatusId());
        ps.setInt(8, user.getUserId());
        return ps.executeUpdate();
    }
```

```
try {
        conn.setAutoCommit(false);

        int result;
        try (PreparedStatement ps =
conn.prepareStatement(sql)) {
            ps.setInt(1, userId);
            result = ps.executeUpdate();
        }

        conn.commit();
        return result;

    } catch (SQLException e) {
        conn.rollback();
        throw e;

    } finally {
        conn.setAutoCommit(true);
    }

try (Statement stmt = conn.createStatement();
         ResultSet rs = stmt.executeQuery(sql)) {

        while (rs.next()) {
            list.add(mapUser(rs));
        }
    }


try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, name);
        ps.setString(2, pwd);

        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() ? mapUser(rs) : null;  // ユーザー
が存在すればUserオブジェクトを返す
        }


try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, userId);

        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() ? mapUser(rs) : null;
        }
    }
```

```
try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, userId);

        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return rs.getString("identity_photo");
            }
        }
    }


try (PreparedStatement checkPs =
conn.prepareStatement(checkSql)) {
        checkPs.setInt(1, userId);

        try (ResultSet rs = checkPs.executeQuery()) {

            if (rs.next()) {
                // 既存 → UPDATE（再申請なので status は
inactive)
                String updateSql = """
                    UPDATE user_identity_verification
                    SET identity_photo = ?,
                        status_id = 1,
                        submitted_at = CURRENT_TIMESTAMP,
                        verified_at = NULL
                    WHERE user_id = ?
                """;

                try (PreparedStatement ps =
conn.prepareStatement(updateSql)) {
                    ps.setString(1, imagePath);
                    ps.setInt(2, userId);
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	SuggestedInfo.java
プログラミング担当	**名前**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	SuggestedInfo-01	SuggestedInfo-02	SuggestedInfo-03
テスト担当者	松田	松田	松田
結果	OK	OK	OK
値がセットされている（Setter）	Y	Y	Y
対応するフィールドを呼び出す（Getter）	suggestedNumber	userId	provisionNumber
セットした値と同じ値が返される	X	X	X
型がint型である	X	X	X

対象となるソースコード

```
SuggestedInfo si = new SuggestedInfo();
si.setSuggestedNumber(100); //Setterで値をセット
int num = si.getSuggestedNumber(); //Getterで値を取得
si.setUserId(50);
int user = si.getUserId();
si.setProvisionNumber(200);
int prov = si.getProvisionNumber();
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	Provision.java
プログラミング担当	**名前**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	Provision-01	Provision-02	Provision-03	Provision-04	Provision-05	Provision-06
テスト担当者	松田	松田	松田	松田	松田	松田
結果	OK	OK	OK	OK	OK	OK
値がセットされている（Setter）	Y	Y	Y	Y	Y	Y
対応する項目を呼び出す（Getter）	provision Number	userId	textbook Number	subjects Number	textbook Detail	textbook Photo
セットした値と同じ値が返される	X	X	X	X	X	X
期待される型通りである	int	int	int	int	string	string

対象となるソースコード

```
Provision p = new Provision();
p.setProvisionNumber(1);
p.setUserId(10);
p.setTextbookNumber(100);
p.setSubjectsNumber(5);
p.setTextbookDetail("書き込みなし、美品です。");
p.setTextbookPhoto("sample.jpg")
```

ウィザー

ウィザー

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	Profile.jsp
プログラミング担当	**名前**
簡単な説明	プロフィール画面

テストNo	ProfileUpdate.jsp	ProfileUpdate.jsp-02
テスト担当者	野中	野中
結果	OK	OK
pass1 === pass2	Y	N

対象となるソースコード

```
if (pass1 !== pass2) {
alert("パスワードが一致しません");
return;
}
```

ウィザー

ウィザー

```
%>
<img
        src="<%=request.getContextPath() + "/" + loginUser.getUsersPhoto()%>"
        style="max-width: 100%; height: auto;">
<%
} else {
%>
<p>画像なし</p>
<%
}
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	Profile.jsp
プログラミング担当	**名前**
簡単な説明	プロフィール画面

テストNo	Profile.jsp-01	Profile.jsp-02
テスト担当者	野中	野中
結果	OK	OK
loginUser.getUsersPhoto() != null && !loginUser.getUsersPhot	Y	N
identityImagePath != null && !identityImagePath.isEmpty()	Y	N

対象となるソースコード

```
if (loginUser.getUsersPhoto() != null && !loginUser.getUsersPhoto().isEmpty()) {
if (identityImagePath != null && !identityImagePath.isEmpty()) {
```

```
%>
<img
        src="<%=request.getContextPath() + "/" + loginUser.getUsersPhoto()%>"
        style="max-width: 100%; height: auto;">
<%
} else {
%>
<p>画像なし</p>
<%
}
}
```

デシジョンテーブル

チーム名	**チームC**
試験対象プログラム名	DbConnect.java
プログラミング担当	**名前**
簡単な説明	データベース接続の生成・取得を行う共通基盤です。

テストNo	waribiki-01	waribiki-02	waribiki-03	waribiki-04
テスト担当者	太郎	次郎	三郎	四郎
結果	OK	OK	OK	OK
adminInfoList = adminListLogic.execute();	Y	N	N	N
会員カード持参	Y	N	Y	Y
金額が3,000円	-	X	-	X
金額が2,500円	X	-	-	-
金額が2,800円	-	-	X	-

対象となるソースコード

```
if (会員カード＝なし) {
    料金＝3,000
} else {
    if (学生証＝あり) {
        料金＝2,500
    } else {
        料金＝2,800
    }
}
```